



AMRITA
VISHWA VIDYAPEETHAM

Advanced protocol engineering and security lab
EXP-4,5,6

Name: Poloju Varshith

Roll no: ch.sc.u4cys23035

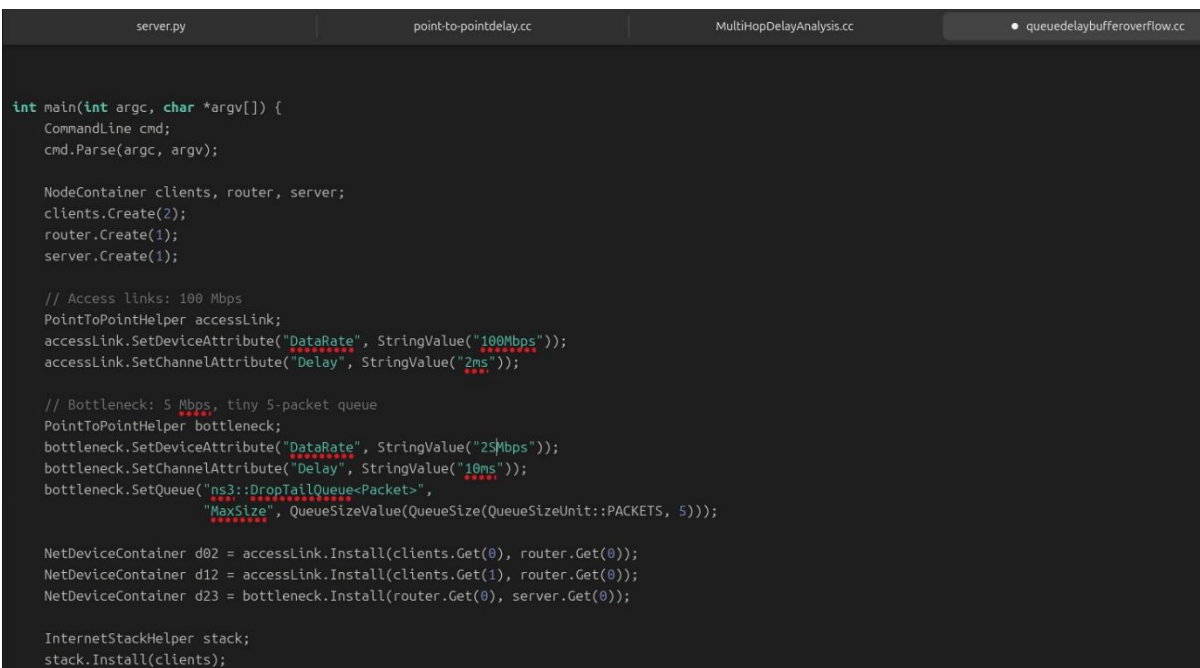
EXPERIMENT 4

AIM: To study the effect of queuing delay and buffer overflow in a network using UDP traffic and observe packet loss under congestion conditions.

DESCRIPTION: In this experiment, UDP traffic is generated across a bottleneck link. Since UDP is a non-adaptive protocol, it does not adjust its sending rate during congestion. As traffic increases, packets get queued in buffers, causing queuing delay. When the buffer limit is exceeded, packet loss occurs due to buffer overflow.

PROGRAM USED: queuedelaybufferoverflow.cc

Program code:

The image shows a code editor with four tabs at the top: 'server.py', 'point-to-pointdelay.cc', 'MultiHopDelayAnalysis.cc', and 'queuedelaybufferoverflow.cc'. The 'queuedelaybufferoverflow.cc' tab is active, displaying C++ code for a network simulation. The code defines a main function that sets up a network topology with clients, a router, and a server. It configures access links with a 100 Mbps data rate and 2ms delay, and a bottleneck link with a 25 Mbps data rate, 10ms delay, and a drop-tail queue. The code then installs these links and sets up an internet stack for the clients.

```
int main(int argc, char *argv[]) {
    CommandLine cmd;
    cmd.Parse(argc, argv);

    NodeContainer clients, router, server;
    clients.Create(2);
    router.Create(1);
    server.Create(1);

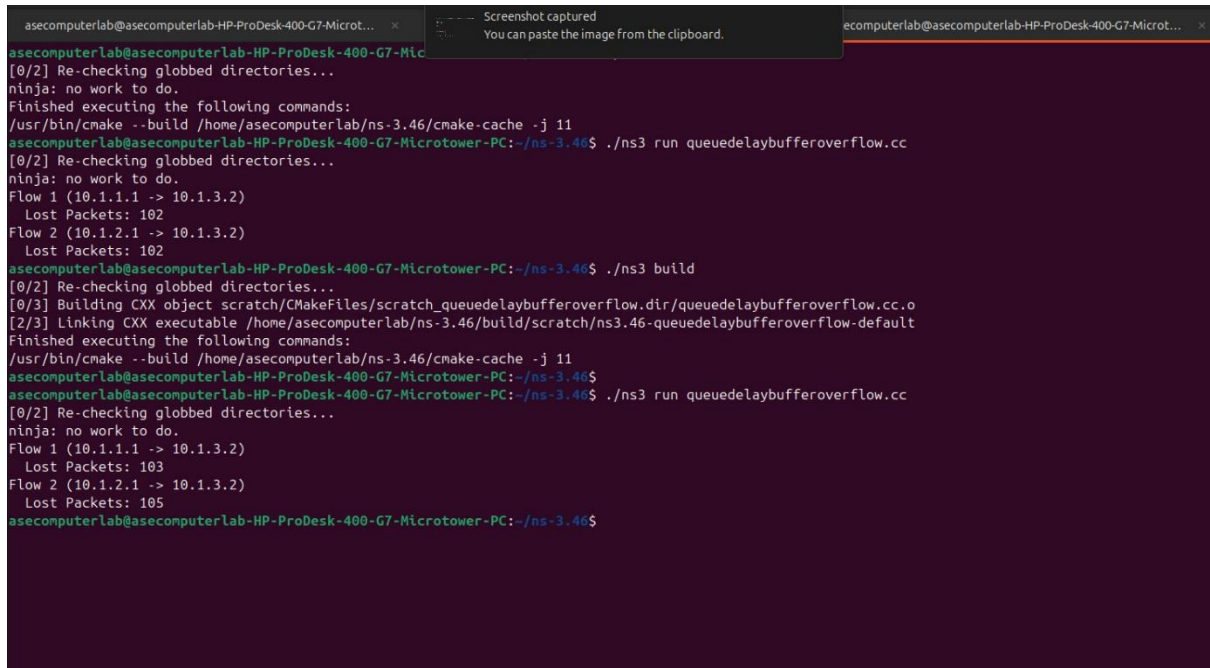
    // Access links: 100 Mbps
    PointToPointHelper accessLink;
    accessLink.SetDeviceAttribute("DataRate", StringValue("100Mbps"));
    accessLink.SetChannelAttribute("Delay", StringValue("2ms"));

    // Bottleneck: 5 Mbps, tiny 5-packet queue
    PointToPointHelper bottleneck;
    bottleneck.SetDeviceAttribute("DataRate", StringValue("25Mbps"));
    bottleneck.SetChannelAttribute("Delay", StringValue("10ms"));
    bottleneck.SetQueue("ns3::DropTailQueue<Packet>",
        "MaxSize", QueueSizeValue(QueueSize(QueueSizeUnit::PACKETS, 5)));

    NetDeviceContainer d02 = accessLink.Install(clients.Get(0), router.Get(0));
    NetDeviceContainer d12 = accessLink.Install(clients.Get(1), router.Get(0));
    NetDeviceContainer d23 = bottleneck.Install(router.Get(0), server.Get(0));

    InternetStackHelper stack;
    stack.Install(clients);
```

Execution:



```
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microt... x  Screenshot captured
You can paste the image from the clipboard.  asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microt... x
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Mic
[0/2] Re-checking globbed directories...
ninja: no work to do.
Finished executing the following commands:
/usr/bin/cmake --build /home/asecomputerlab/ns-3.46/cmake-cache -j 11
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$ ./ns3 run queuedelaybufferoverflow.cc
[0/2] Re-checking globbed directories...
ninja: no work to do.
Flow 1 (10.1.1.1 -> 10.1.3.2)
  Lost Packets: 102
Flow 2 (10.1.2.1 -> 10.1.3.2)
  Lost Packets: 102
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$ ./ns3 build
[0/2] Re-checking globbed directories...
[0/3] Building CXX object scratch/CMakeFiles/scratch_queuedelaybufferoverflow.dir/queuedelaybufferoverflow.cc.o
[2/3] Linking CXX executable /home/asecomputerlab/ns-3.46/build/scratch/ns3.46-queuedelaybufferoverflow-default
Finished executing the following commands:
/usr/bin/cmake --build /home/asecomputerlab/ns-3.46/cmake-cache -j 11
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$ ./ns3 run queuedelaybufferoverflow.cc
[0/2] Re-checking globbed directories...
ninja: no work to do.
Flow 1 (10.1.1.1 -> 10.1.3.2)
  Lost Packets: 103
Flow 2 (10.1.2.1 -> 10.1.3.2)
  Lost Packets: 105
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$
```

Result:

Thus, the behavior of queuing delay and buffer overflow in UDP traffic was successfully studied

EXPERIMENT 5

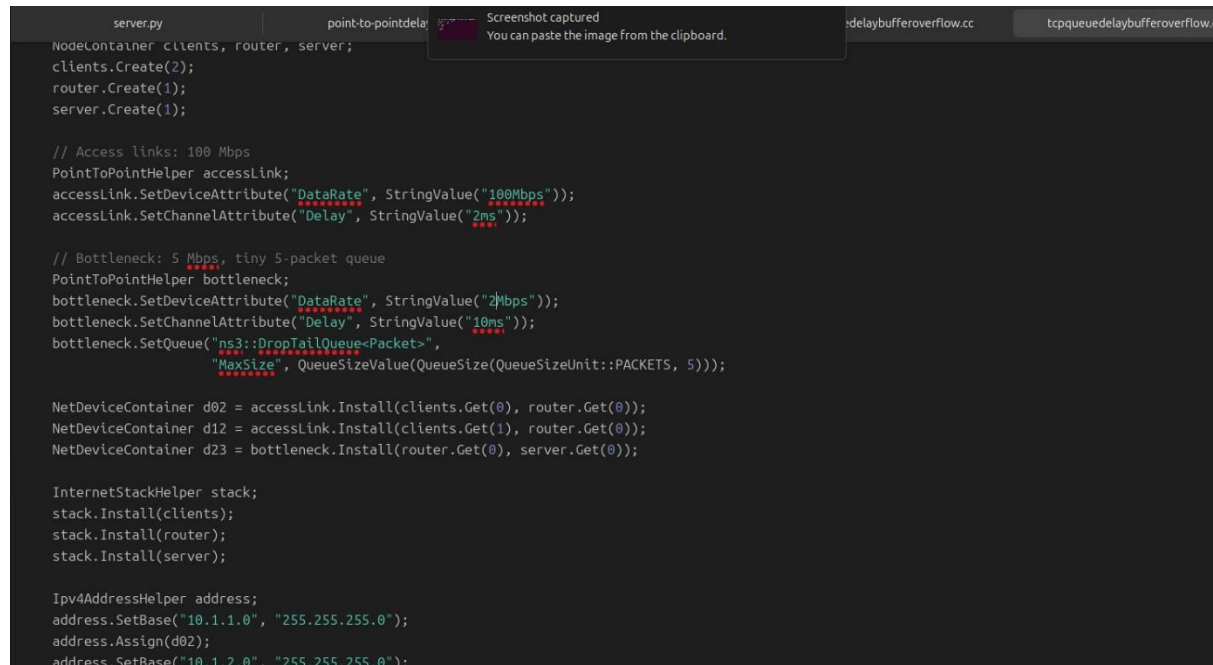
QUEING DELAY AND BUFFER OVERFLOW -TCP

AIM: To analyze queuing delay and buffer overflow in a network using TCP traffic and observe congestion control behavior.

DESCRIPTION: In this experiment, TCP traffic is transmitted across a shared bottleneck link. TCP is an adaptive protocol that dynamically adjusts its congestion window based on network conditions. When congestion occurs, TCP reduces its sending rate to minimize packet loss.

tcpqueuedelaybufferoverflow.cc

Program code:



```
server.py      point-to-pointdelay  Screenshot captured
You can paste the image from the clipboard.  delaybufferoverflow.cc  tcpqueuedelaybufferoverflow.cc

NodeContainer clients, router, server;
clients.Create(2);
router.Create(1);
server.Create(1);

// Access links: 100 Mbps
PointToPointHelper accessLink;
accessLink.SetDeviceAttribute("DataRate", StringValue("100Mbps"));
accessLink.SetChannelAttribute("Delay", StringValue("2ms"));

// Bottleneck: 5 Mbps, tiny 5-packet queue
PointToPointHelper bottleneck;
bottleneck.SetDeviceAttribute("DataRate", StringValue("5Mbps"));
bottleneck.SetChannelAttribute("Delay", StringValue("10ms"));
bottleneck.SetQueue("ns3::DropTailQueue<Packet>",
    "MaxSize", QueueSizeValue(QueueSize(QueueSizeUnit::PACKETS, 5)));

NetDeviceContainer d02 = accessLink.Install(clients.Get(0), router.Get(0));
NetDeviceContainer d12 = accessLink.Install(clients.Get(1), router.Get(0));
NetDeviceContainer d23 = bottleneck.Install(router.Get(0), server.Get(0));

InternetStackHelper stack;
stack.Install(clients);
stack.Install(router);
stack.Install(server);

Ipv4AddressHelper address;
address.SetBase("10.1.1.0", "255.255.255.0");
address.Assign(d02);
address.SetBase("10.1.2.0", "255.255.255.0");
```

Execution:

1st = Bottleneck link: 5 Mbps

2nd = Bottleneck link: 2 Mbps

```
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~$ cd ns-3.46
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$ ./ns3 build
[0/2] Re-checking globbed directories...
ninja: no work to do.
Finished executing the following commands:
/usr/bin/cmake --build /home/asecomputerlab/ns-3.46/cmake-cache -j 11
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$ ./ns3 run tcpqueuedelaybufferoverflow.cc
[0/2] Re-checking globbed directories...
ninja: no work to do.
Flow 1 (10.1.1.1 -> 10.1.3.2)
  Lost Packets: 27
Flow 2 (10.1.2.1 -> 10.1.3.2)
  Lost Packets: 24
Flow 3 (10.1.3.2 -> 10.1.1.1)
  Lost Packets: 0
Flow 4 (10.1.3.2 -> 10.1.2.1)
  Lost Packets: 0
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$
```

```
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$ ./ns3 build
[0/2] Re-checking globbed directories...
[0/3] Building CXX object scratch/CMakeFiles/scratch_tcpqueuedelaybufferoverflow.dir/tcpqueuedelaybufferoverflow.cc.o
[2/3] Linking CXX executable /home/asecomputerlab/ns-3.46/build/scratch/ns3.46-tcpqueuedelaybufferoverflow-default
Finished executing the following commands:
/usr/bin/cmake --build /home/asecomputerlab/ns-3.46/cmake-cache -j 11
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$ ./ns3 run tcpqueuedelaybufferoverflow.cc
[0/2] Re-checking globbed directories...
ninja: no work to do.
Flow 1 (10.1.1.1 -> 10.1.3.2)
  Lost Packets: 18
Flow 2 (10.1.2.1 -> 10.1.3.2)
  Lost Packets: 18
Flow 3 (10.1.3.2 -> 10.1.1.1)
  Lost Packets: 0
Flow 4 (10.1.3.2 -> 10.1.2.1)
  Lost Packets: 0
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$
```

Result:

Thus, queuing delay and buffer overflow behavior in TCP traffic was successfully analyzed.

Experiment 6

Aim: To study and compare the behavior of adaptive TCP traffic and non-adaptive UDP traffic over a shared bottleneck link by observing TCP congestion window dynamics, throughput, and packet loss under congestion.

Description:

This experiment compares TCP and UDP traffic when both share a congested network link. TCP adapts its transmission rate based on congestion feedback, whereas UDP continues sending packets without adaptation, leading to unfair bandwidth usage and packet loss.

tcpvsudp.cc

Program code:

```
clients.Create(2);
router.Create(1);
server.Create(1);

// Access links: 100 Mbps
PointToPointHelper accessLink;
accessLink.SetDeviceAttribute("DataRate", StringValue("100Mbps"));
accessLink.SetChannelAttribute("Delay", StringValue("2ms"));

// Bottleneck link: 5 Mbps, 5-packet queue
PointToPointHelper bottleneck;
bottleneck.SetDeviceAttribute("DataRate", StringValue("5Mbps"));
bottleneck.SetChannelAttribute("Delay", StringValue("10ms"));
bottleneck.SetQueue("ns3::DropTailQueue<Packet>",
    "MaxSize", QueueSizeValue(QueueSize(QueueSizeUnit::PACKETS, 5)));

NetDeviceContainer d02 = accessLink.Install(clients.Get(0), router.Get(0));
NetDeviceContainer d12 = accessLink.Install(clients.Get(1), router.Get(0));
NetDeviceContainer d23 = bottleneck.Install(router.Get(0), server.Get(0));

InternetStackHelper stack;
stack.Install(clients);
stack.Install(router);
stack.Install(server);

Ipv4AddressHelper address;
address.SetBase("10.1.1.0", "255.255.255.0");
address.Assign(d02);
address.SetBase("10.1.2.0", "255.255.255.0");
address.Assign(d12);
```

Execution:

1st = Bottleneck link: 5 Mbps

2nd = Bottleneck link: 50 Mbps

```
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$ ./ns3 build
[0/2] Re-checking globbed directories...
[0/3] Building CXX object scratch/CMakeFiles/scratch_tcpvsudp.dir/tcpvsudp.cc.o
[2/3] Linking CXX executable /home/asecomputerlab/ns-3.46/build/scratch/ns3.46-tcpvsudp-default
Finished executing the following commands:
/usr/bin/cmake --build /home/asecomputerlab/ns-3.46/cmake-cache -j 11
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$ ./ns3 run tcpvsudp.cc
[0/2] Re-checking globbed directories...
ninja: no work to do.

^[[A^[Flow 1 (10.1.1.1 -> 10.1.3.2) Throughput: 16.0379 Mbps, Lost packets: 8
Flow 2 (10.1.2.1 -> 10.1.3.2) Throughput: 16.023 Mbps, Lost packets: 8
Flow 3 (10.1.2.1 -> 10.1.3.2) Throughput: 17.4838 Mbps, Lost packets: 1907
Flow 4 (10.1.3.2 -> 10.1.1.1) Throughput: 0.743176 Mbps, Lost packets: 0
Flow 5 (10.1.3.2 -> 10.1.2.1) Throughput: 0.742269 Mbps, Lost packets: 0
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$
```

```
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~$ cd ns-3.46
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$ ./ns3 build
[0/2] Re-checking globbed directories...
ninja: no work to do.
Finished executing the following commands:
/usr/bin/cmake --build /home/asecomputerlab/ns-3.46/cmake-cache -j 11
asecomputerlab@asecomputerlab-HP-ProDesk-400-G7-Microtower-PC:~/ns-3.46$ ./ns3 run tcpvsudp.cc
[0/2] Re-checking globbed directories...
ninja: no work to do.
Flow 1 (10.1.1.1 -> 10.1.3.2) Throughput: 1.69937 Mbps, Lost packets: 38
Flow 2 (10.1.2.1 -> 10.1.3.2) Throughput: 1.53789 Mbps, Lost packets: 31
Flow 3 (10.1.2.1 -> 10.1.3.2) Throughput: 1.74379 Mbps, Lost packets: 3776
Flow 4 (10.1.3.2 -> 10.1.1.1) Throughput: 0.0964463 Mbps, Lost packets: 0
Flow 5 (10.1.3.2 -> 10.1.2.1) Throughput: 0.0914326 Mbps, Lost packets: 0
```

Result:

Thus, the behavior of adaptive TCP traffic and non-adaptive UDP traffic over a shared bottleneck link was successfully studied and compared.